

งานวิจัยที่เกี่ยวข้องกับแบบรูป (Research in Design Patterns)

นักวิจัยศึกษาประเด็นต่าง ๆ ที่เกี่ยวข้องกับ design patterns ดังนี้

1. ผลกระทบต่อคุณภาพซอฟต์แวร์

คุณภาพของซอฟต์แวร์ เช่น ความเร็วในการประมวลผล (runtime) ความคงเส้นคงวา (stability) ความสามารถในการบำรุงรักษา (maintainability) และการปรับปรุงคุณภาพโดยรวมของซอฟต์แวร์ จากการศึกษพบว่าการใช้ patterns สามารถเพิ่มความสามารถในการจัดการกับปัญหาที่ซับซ้อนในกระบวนการพัฒนาได้ อย่างไรก็ตาม ผลลัพธ์ยังเป็นที่ถกเถียงและบางงานได้ผลลัพธ์ที่ไม่สอดคล้องกัน เนื่องจากความแตกต่างของตัวชี้วัดและมุมมองที่ใช้ในการประเมินคุณภาพซอฟต์แวร์

Lindung Parningotan Manik, Hani Febri Mustika, Arida Ferti Syafiandini [1] introduced in the early stage of software development where model-driven architecture is used as the engineering approach. A RESTful internet payment gateway API (Application Programming Interface) wrapper is selected as the case study.

To evaluate the impacts of implementation, static analysis is used to examine the generated code before and after adding the design patterns. The result shows that the design decision increases cohesion, complexity, coupling, inheritance, and size metrics of the source codes.

Analysis:

five software metrics are measured by using static analysis to investigate the impact of the design patterns implementation

1. Cohesion metric - represented by the LCOM5 (Lack of Cohesion in Methods 5), in measuring to what extent the source code elements are coherent in the system
2. Complexity metric - represented by the WMC (Weighted Methods per Class), in measuring the complexity of source code elements
3. Coupling metric - represented by the CBO (Coupling Between Object classes), in measuring the amount of interdependencies of source code elements
4. Inheritance metric - represented by DIT (Depth of Inheritance Tree), in measuring the aspects of the inheritance hierarchy of the system
5. Size metric - represented by the LLOC (Logical Lines of Code), in measuring the basic properties of the analyzed system in terms of cardinalities

👉 ออกแบบ RESTful payment gateway API แล้ววิเคราะห์ด้วย 5 metrics

Carmine Gravino and Michele Risi [2] verified that the use of design pattern impacts on the quality of the software.

👉 ยืนยันว่า design pattern มีผลต่อคุณภาพซอฟต์แวร์

Muhammad Noman Riaz [3] surveyed existing software design patterns and concluded how GoF design patterns impact software quality in terms of maintainability, evolution, performance, and faults. According to the finding, software maintainability is provided by Abstract pattern, Composite pattern and Decorator pattern.

👉 สรุปว่า Abstract pattern, Composite pattern และ Decorator pattern สนับสนุน software maintainability

Bruno L. Sousa, et.al. [4] investigated large classes in five Java projects developed with design patterns. They concluded that Composite pattern and Factory Method pattern have a low co-occurrence with long method whereas Template method and Observer method have a high co-occurrence with large class and long method.

👉 วิเคราะห์คลาสขนาดใหญ่ใน 5 projects ที่ใช้ design patterns พบว่า
Composite pattern & Factory Method pattern มี low co-occurrence
Template method & Observer method มี high co-occurrence

Apostolos Ampatzoglou, et.al. [5] investigated the effect of GoF design patterns on class stability. The results suggest that the application of design patterns can provide the expected “shielding” of certain pattern-participating classes against changes, depending on their role in the pattern. Because less stable classes are expected to require more effort while testing, and urge for refactoring activities.

Different GoF design patterns provide different levels of stability to the classes that participate in them. For example, Singleton, FaçadeMediator, Observer, Composite, and Decorator occurrences seem to consist of classes that are more resistant to changes propagating from other classes.

The Aggregate role is less stable than classes that play the Component role. On the other hand, in design patterns that involve inheritance, public Superclasses are more stable than Subclasses.

👉 ศึกษาผลกระทบของ GoF ที่มีต่อ class stability พบว่า

- Singleton, FaçadeMediator, Observer, Composite, and Decorator ทนต่อการเปลี่ยนแปลงของคลาสอื่นที่เกี่ยวข้องได้มากกว่า pattern แบบอื่น

Stefan Nadschläger and Josef Küng [6] analyzed existing knowledge processing systems on usage of Gang-of-Four design patterns and where to apply them. The design patterns were identified manually in a structured analysis by reverse engineering the source code, supported by a design pattern detection tool. As a result, Gang-of-Four design patterns, suitable for developing custom knowledge processing systems, are presented and discussed.

👉 พบว่า GoF เหมาะกับ knowledge processing system

Daniel Feitosa, et.al. [7] analyzed five open-source projects in order to study if the presence of patterns enforces the conformance to good coding practices. The obtained results suggest that classes not participating in patterns are more probable to violate good coding practices for correctness, performance and security. We observed that patterns with more complex structure (e.g., Decorator) and pattern roles that are more change-prone (e.g., Subclasses) are more likely to be associated with a higher number of violations (up to 50 times more violations).

This finding implies that investing in a well-thought architecture based on best practices, such as patterns, is often accompanied with cleaner code with fewer violations.

👉 ศึกษา 5 open-source projects แล้วยืนยันว่าซอฟต์แวร์ที่มี architecture ดี ๆ สร้างโดย based on best practices อย่างเช่น design patterns จะทำให้ได้ cleaner code with fewer violations

Sofia Charalampidou, et.al. [8] proposed a theoretical model for understanding the effect of patterns on quality. The results suggest that Decorator instances that are not expected to evolve through the addition of components in composite objects decrease system cohesion and therefore, modularity and maintainability are weakened.

👉 พบว่าการใช้ Decorator pattern ที่ไม่เกี่ยวข้องกับ components ใน composite objects ส่งผลในด้านลบ คือทำให้ลด modularity & maintainability

The process of building a WWW document is done fully on the client-side. To ensure the best possible user experience it should be as short as possible. Artur Zochniak and Tomasz Walkowiak [9] present performance comparison of observer design pattern implementations in two JavaScript frameworks: AngularJS, EmberJS. They tested their performance in a function of size of input data. The presented results allow to build fast response web pages.

👉 เปรียบเทียบ performance ของ AngularJS เทียบกับ EmberJS ที่เขียนเป็น Observer pattern เหมือนกัน โดยเขียน scripts เพื่อ gen Web page deploy บน Heroku & เขียน loop อ่าน dataset จำนวน 100 ไปจนถึง 50,000 elements แล้ววัด rendering time พบว่า EmberJS เร็วกว่าเกือบ 2 เท่า เมื่อ dataset มากขึ้น

Mert Ozkaya and Mehmet Alp Kose [12] enhanced learning design patterns by creating the software design toolset, called DesPat. They included the factory, composite, facade, observer, singleton, and visitor design patterns in the toolset which allows the computer science students to draw a class diagram and check for correctness.

👉 พัฒนาซอฟต์แวร์เพื่อสนับสนุนการเรียนรู้ design patterns

Vitalii Velychko [13] describes the process of designing a flexible architectural solution for semantic comparison of texts based on the Visitor design pattern. They concluded that implementation of the Visitor design pattern enhances the semantic texts comparison software systems such as high recognition accuracy, estimation of different comparison of multiple algorithms.

👉 สรุปว่า Visitor pattern สนับสนุนซอฟต์แวร์ที่ใช้เปรียบเทียบ semantic text

D. C. Bree and M. Ó. Cinnéide [11] studied the impact of the Visitor pattern on energy efficiency and experimentally investigated the energy impact of varying the implementation of this pattern. They found that the energy consumption reductions of greater than 7% in the textbook example when they removed the Visitor pattern from the software. [visitor02]

👉 ศึกษาผลกระทบของ Visitor pattern ที่มีต่อการใช้พลังงานของเครื่องคอมพิวเตอร์ พบว่าเมื่อเอา Visitor pattern ออกไปจากซอฟต์แวร์ ทำให้ลดการใช้พลังงานลง 7%

Amal Khairin, Dana Sulistyio Kusumo, Yudi Priyadi [14] analyzed the impact of design patterns on mobile application performance. They found that design patterns can affect application performance depending on the design pattern used. The Strategy pattern and Visitor pattern optimize memory usage by 1%, while the Bridge pattern increases memory usage by 2%.

👉 พบว่า Strategy pattern & Visitor pattern ช่วยลดการใช้งานหน่วยความจำลง 1% ในขณะที่ Bridge pattern เพิ่มการใช้งานหน่วยความจำขึ้น 2%

References

1. Lindung Parningotan Manik, Hani Febri Mustika, Arida Ferti Syafiandini. Design Pattern Evaluation on A RESTful API Wrapper: A Case Study of Software Integration with An Internet Payment Gateway using Model-Driven Architecture. Journal of Information Technology and Computer Science Volume 4, Number 3, Desember 2019, pp. 222-232
2. Carmine Gravino and Michele Risi. **How the use of design patterns affects the quality of software systems: a preliminary investigation.** 43rd Euromicro Conference on Software Engineering and Advanced Applications, 2017.
3. Muhammad Noman Riaz. **Impact of Software Design Patterns on the Quality of Software : A Comparative Study.** International Conference on Computing, Mathematics and Engineering Technologies – iCoMET 2018.
4. Brazilian Symposium on Information Systems, Lavras, Minas Gerais, June 5-8, 2017
5. Apostolos Ampatzoglou, et.al. The Effect of GoF Design Patterns on Stability: A Case Study. IEEE Transactions on Software Engineering Volume: 41 , Issue: 8 , Aug. 1 2015.

6. Stefan Nadschläger and Josef Küng. Analysis of GoF Design Patterns used in Knowledge Processing Systems. Proceedings of the 22nd European Conference on Pattern Languages of Programs EuroPLoP 2017.
7. Daniel Feitosa, et.al. **What can violations of good practices tell about the relationship between GoF patterns and run-time quality attributes?** Information and Software Technology 105 (2019) 1–16.
8. Sofia Charalampidou, et.al. A theoretical model for capturing the impact of design patterns on quality: the decorator case study. Proceedings of the Symposium on Applied Computing SAC'17 Pages 1231-1238.
9. Artur Zochniak and Tomasz Walkowiak. Performance Comparison of Observer Design Pattern Implementations in JavaScript. International Conference on Reliability and Statistics in Transportation and Communication pp 466-475, Lecture Notes in Networks and Systems 36, 2018.
10. Déaglán Connolly Bree and Mel Ó Cinnéide, "The Energy Cost of the Visitor Pattern," 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME), Limassol, Cyprus, 2022, pp. 317-328
11. Ozkaya, M., & Kose, M. A. (2021). DesPat: A modeling toolset for designing and implementing software systems using design patterns. In ENASE (251-260).
12. Vitalii Velychko. Architectural Solution of System for Semantic Comparison of the Texts. International Journal Information Theories and Applications, Vol.26, Number 4, 2019.
13. Khairin, A., Kusumo, D., & Priyadi, Y. (2022). Analysis of The Impact of Software Detailed Design on Mobile Application Performance Metrics. Building of Informatics, Technology and Science (BITS), 4(1), 226–234. <https://doi.org/10.47065/bits.v4i1.1739>

2. การออกแบบระบบที่ใช้ AI

การศึกษาการประยุกต์ใช้ design patterns ในระบบที่มีการใช้ปัญญาประดิษฐ์ (AI) ด้วยการสร้างฐานข้อมูลของ patterns ที่เหมาะสมสำหรับระบบ AI เช่น ระบบ machine learning และการพัฒนาซอฟต์แวร์ที่ใช้ AI ซึ่งช่วยให้นักพัฒนาสามารถนำ patterns ที่ดีที่สุดมาใช้เพื่อเพิ่มประสิทธิภาพการออกแบบ

- การพัฒนา AI-Based Systems ด้วย Design Patterns เช่น การออกแบบโครงสร้างระบบที่ใช้ Machine Learning, ระบบ Recommendation และการจัดการข้อมูลขนาดใหญ่
- การศึกษาว่า patterns ใดที่ช่วยเพิ่มประสิทธิภาพในกระบวนการพัฒนา Machine Learning เช่น การจัดการข้อมูล การปรับแต่งโมเดล และการประเมินผล ตัวอย่างของ patterns ที่ถูกนำไปใช้บ่อย ได้แก่ Factory Pattern สำหรับสร้างโมเดล, Strategy Pattern สำหรับเลือกวิธีการประมวลผล และ Observer Pattern สำหรับการติดตามการเปลี่ยนแปลงของข้อมูลหรือผลลัพธ์
- การประเมินและปรับปรุงคุณภาพของ AI Systems: งานวิจัยบางส่วนมุ่งเน้นที่การประเมินผลกระทบของ design patterns ต่อคุณภาพของระบบ AI เช่น ความสามารถในการบำรุงรักษา ความสามารถในการขยายระบบ และการปรับตัวของระบบ
- การพัฒนา repository ของ design patterns สำหรับ AI ช่วยให้นักพัฒนาสามารถเข้าถึงและเลือกใช้ patterns ที่เหมาะสมกับการพัฒนาระบบได้ง่ายขึ้น เช่น การใช้ pattern-specific templates เพื่อสร้าง AI components ที่ซับซ้อน

3. การตรวจจับและการปรับใช้ patterns

มีการพัฒนาเครื่องมือและเทคนิคที่ช่วยในการตรวจจับ patterns ในซอฟต์แวร์เพื่อประเมินผลกระทบของ patterns ต่อการออกแบบและการพัฒนา การวิจัยเหล่านี้ช่วยปรับปรุงกระบวนการระบุและเลือกใช้ patterns ที่เหมาะสมที่สุดสำหรับแต่ละสถานการณ์การพัฒนา

Derezińska A., Byczkowski M. [12] conducts experiments on DP utilization in C# programs According to project-level analysis, DP increase maintainability of software. Class-level analysis of programs with DP approved high encapsulation of DP code. The experiments have been conducted on 20 real programs & use C# Analyzer for Detection of Design Patterns

Result:

In general, differences between code with and without DP were more visible at the project level than at the class level. One of possible reasons could be the fact that the projects were developed by various programmers. Those developers who knew DP and applied them might have been able to implement high quality code that would be easy to comprehend and maintain

 ใช้ analyzer tool detect design pattern ของ 20 real-world applications

References

1. Derezińska A., Byczkowski M. (2020) Evaluation of Design Pattern Utilization and Software Metrics in C# Programs. Advances in Intelligent Systems and Computing, vol 987. Springer, Cham, 2019

4. การนำเสนอแบบรูปใหม่

In terms of Industrial IoT, Gedare Bloom, et.al. [1] identified common input-output 6 design patterns for control systems: Closed-loop, Cloud-in-the-Loop, Open-Loop, Cloud-on-the-Loop, Publisher, and Device-to-Device. Each pattern supports the particular purpose of the industrial control system. For example, Publisher pattern facilitates data collection from field devices whereas the Device-to-Device pattern enhances shared data among peer machines.

 ออกแบบ 6 patterns เพื่อใช้ในงาน industrial IoT เช่น

- Device-to-Device pattern เพื่อ share data among peer machines
- Publisher pattern เพื่อช่วยรวบรวมข้อมูลจาก field devices

In terms of Smart Contract in the Ethereum Ecosystem, Maximilian Wohrer and Uwe Zdun [2] categorized widely used 18 design patterns into 5 categories: Action and Control, Authorization, Lifecycle, Maintenance, and Security. For example, authorization consists of Ownership pattern and Access Restriction pattern.

 แบ่ง 18 patterns ที่ใช้กับ smart contract ใน Ethereum Ecosystem อย่างแพร่หลาย เป็น 5 ประเภท ได้แก่ Action and Control, Authorization, Lifecycle, Maintenance, and Security

References

1. Gedare Bloom, et.al. **Design Patterns for the Industrial Internet of Things**. 14th IEEE International Workshop on Factory Communication Systems (WFCS) 2018.
2. Maximilian Wohrer and Uwe Zdun. **Design Patterns for Smart Contracts in the Ethereum Ecosystem**. IEEE International Conference on Blockchain 2018.